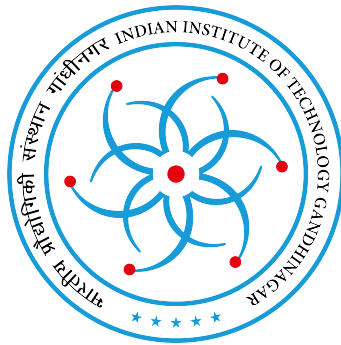

Indian Institute of Technology Gandhinagar



Semantic Search Engine

Dense Retrieval, Hybrid Ranking,
Approximate Nearest Neighbour Indexing

Submitted by

Yash Goyal (24110399)	Computer Science and Engineering
Garv Singhal (24110119)	Computer Science and Engineering
Chaitanya Yogesh Bhoite (24110086)	Artificial Intelligence
Harsh Kumar Jha (24110130)	Artificial Intelligence

Junior Undergraduate

Under the guidance of

Prof. Anirban Dasgupta

Professor, Department of Computer Science and Engineering
IIT Gandhinagar

Contents

Abstract	4
1 Introduction	4
1.1 Motivation	4
1.2 Project Scope and Goals	5
1.3 Contributions	5
2 Problem Statement	6
3 Background and Theory	7
3.1 BM25: Best Match 25	7
3.2 Dense Retrieval and Bi-Encoders	7
3.3 Cross-Encoder Reranking	7
3.4 Approximate Nearest Neighbour Search and HNSW	8
3.5 Word2Vec	8
3.6 Evaluation Metrics	8
4 System Architecture	9
4.1 Overview	9
4.2 Offline Indexing Pipeline	9
4.3 Online Query Pipeline	9
4.4 Custom C++ HNSW Module	10
4.5 Streamlit Application	10
5 Datasets	11
5.1 MS MARCO v1.1	11
5.2 SciFact (BEIR)	11
6 Data Preprocessing	12
6.1 MS MARCO Preprocessing	12
6.2 BM25 Tokenisation	13
6.3 SciFact Preprocessing	13
6.4 Caching Strategy	13
7 Methodology	13
7.1 Embedding Generation	13
7.2 FAISS Index Construction	14
7.3 BM25 Index	14
7.4 Hybrid Retrieval	14
7.5 Cross-Encoder Reranking	15

7.6	Custom HNSW Index	15
7.7	Word2Vec Embeddings	15
7.8	Evaluation Protocol	15
8	Experiments	16
8.1	Experiment 1: BM25 vs. BERT vs. Hybrid on MS MARCO	16
8.1.1	Setup	16
8.1.2	Procedure	16
8.2	Experiment 2: ANN vs. Exact Search Timing	16
8.2.1	Setup	16
8.2.2	Procedure	16
8.3	Experiment 3: Query Type Analysis	17
8.3.1	Setup	17
8.3.2	Procedure	17
8.4	Experiment 4: Cross-Encoder Reranking on SciFact	17
8.4.1	Setup	17
8.4.2	Procedure	17
8.5	Experiment 5: Word2Vec vs. BERT on SciFact	18
8.5.1	Setup	18
8.6	Experiment 6: Custom HNSW vs. FAISS HNSW	18
8.6.1	Setup	18
8.6.2	Procedure	18
9	Results and Analysis	18
9.1	BM25 vs. BERT vs. Hybrid (MS MARCO)	18
9.2	ANN vs. Exact Search Timing	19
9.3	Query Type Analysis	20
9.4	Cross-Encoder Reranking on SciFact	20
9.5	Word2Vec vs. BERT on SciFact	20
9.6	Custom HNSW ef_search Sweep	21
10	Discussion	21
10.1	When Does BM25 Compete with BERT?	21
10.2	Why Hybrid Retrieval Does Not Always Win	21
10.3	HNSW Hyperparameter Sensitivity	21
10.4	SciFact Domain Generalisation	21
10.5	Correctness of the Custom HNSW Implementation	22
11	Limitations	22
11.1	Subset Size for Evaluation	22
11.2	Fixed BM25 Hyperparameters	22

11.3 Fixed Hybrid Alpha	22
11.4 HNSW Disk Persistence	22
12 Future Scope	23
12.1 Domain-Adaptive Fine-Tuning	23
12.2 Learned Sparse Retrieval (SPLADE)	23
12.3 ColBERT Late Interaction	23
12.4 HNSW Disk Persistence and Incremental Insertion	23
12.5 Larger-Scale Evaluation	23
13 Conclusion	23
References	24

Abstract

This report presents a comprehensive study of a Semantic Search Engine built as a unified end-to-end system. The project constructs and evaluates multiple retrieval pipelines on two established information-retrieval benchmarks: the MS MARCO v1.1 passage-ranking dataset and the SciFact scientific fact-verification corpus from the BEIR benchmark. Three retrieval strategies are implemented and compared: BM25 (classical lexical search), BERT-based dense retrieval (semantic vector search), and a Hybrid approach that fuses both signals. A two-stage retrieve-and-rerank pipeline using a cross-encoder is also incorporated to demonstrate precision gains over first-stage retrieval alone. As an additional baseline, a Word2Vec embedding model is trained on the SciFact corpus to contextualise the contribution of contextual language representations. Most notably, a complete Hierarchical Navigable Small World (HNSW) approximate nearest-neighbour index is implemented from scratch in C++ and exposed to Python via pybind11, replacing the FAISS library for ANN search in the second experimental phase. Performance is evaluated using Precision@ k and Recall@ k across multiple retrieval depths. The system is delivered through an interactive Streamlit web application supporting query search, side-by-side model comparison, and PCA-based embedding visualisation. Together, the components form a research-grade exploration of modern information retrieval, from classical keyword scoring to neural embeddings and hand-crafted graph-based indexing.

1 Introduction

1.1 Motivation

The ability to retrieve relevant information in response to a natural-language query is one of the most practically important problems in computer science. Modern users do not formulate their information needs in rigid boolean syntax; they ask natural questions, use paraphrases, and expect systems to infer intent from meaning rather than surface form. Classical retrieval algorithms, while computationally efficient and highly interpretable, are fundamentally limited to matching query and document at the word level. The gap between what a user *says* and what a relevant document *contains* is known as the vocabulary mismatch problem, and it is the central motivation for the semantic search paradigm.

Consider a concrete example. A user queries “*ways to treat rhinorrhea*”, but the most relevant document in the corpus describes “*how to stop a running nose*”. A keyword-based retrieval system finds zero term overlap and assigns the document a score of zero. A human reader, however, immediately recognises that both phrases convey the same information. Closing this gap requires a system that understands text at the level of meaning rather than surface form.

The rise of large pre-trained language models, particularly BERT and its derivatives, has made it practically feasible to encode sentences into dense vector representations where semantic similarity corresponds directly to geometric proximity. This report documents the design, implementation, and evaluation of a system that exploits these representations for retrieval, compares them rigorously with the classical BM25 baseline, and investigates a variety of architectural enhancements.

1.2 Project Scope and Goals

The project is built around the following concrete objectives:

- Load and preprocess two real-world retrieval benchmarks—MS MARCO v1.1 and SciFact—and construct ground-truth relevance sets for evaluation.
- Implement BM25 lexical retrieval, BERT bi-encoder dense retrieval, and a normalised hybrid fusion of the two.
- Build and compare two vector index types: an exact brute-force inner-product index and a Hierarchical Navigable Small World (HNSW) approximate nearest-neighbour (ANN) index, first via FAISS and then via a custom C++ implementation.
- Extend the retrieval pipeline with a cross-encoder reranking stage and a Word2Vec embedding baseline.
- Evaluate all methods with Precision@ k and Recall@ k at multiple retrieval depths.
- Deliver an interactive Streamlit application supporting query search, model comparison, and embedding visualisation.

1.3 Contributions

The primary contributions of this project are as follows:

- **Unified retrieval framework:** a single codebase supporting BM25, BERT ANN, BERT Exact, and Hybrid retrieval with a shared evaluation harness.
- **Multi-dataset evaluation:** systematic experimentation on both a large general-domain corpus (MS MARCO) and a small specialised scientific corpus (SciFact).
- **Two-stage reranking:** integration of a cross-encoder model as a second-stage reranker on top of first-stage ANN retrieval.
- **Custom C++ HNSW index:** a complete from-scratch implementation of the HNSW data structure, compiled as a Python extension via pybind11, with configurable hyperparameters and batch query support.

- **Comparative embedding study:** direct comparison of BERT contextual embeddings against Word2Vec averaged embeddings on the same corpus and evaluation set.
- **Interactive deployment:** a Streamlit application with multiple analytical views suitable for both technical and non-technical audiences.
- **Public implementation and deployment:** the full source code and deployed demonstration are made available for reproducibility and interactive testing. The GitHub repository contains the preprocessing pipeline, retrieval modules, evaluation scripts, custom C++ HNSW implementation, and Streamlit frontend, while the Streamlit deployment allows users to test the search engine directly through a browser.
 - **GitHub Repository:** [GitHub Link](#)
 - **Streamlit Demo:** [Streamlit App](#)

2 Problem Statement

The formal problem addressed in this project is passage retrieval: given a query q and a corpus of documents $\mathcal{D} = \{d_1, d_2, \dots, d_N\}$, produce a ranked list of the top- k documents most relevant to q . Relevance is defined by human-annotated ground-truth labels available in the benchmark datasets used.

A retrieval model is a function $f : (q, \mathcal{D}) \rightarrow [(d_{r_1}, s_1), (d_{r_2}, s_2), \dots, (d_{r_k}, s_k)]$ that assigns a relevance score s_i to each document and ranks them in descending score order. The central challenge is designing f such that documents semantically relevant to q receive high scores, even when they share few or no surface-level words with q .

The project investigates three families of f :

1. **Lexical retrieval (BM25):** f scores documents by term-frequency and inverse-document-frequency statistics. Fast and interpretable, but blind to synonyms and paraphrases.
2. **Dense retrieval (BERT bi-encoder):** f encodes q and each d_i independently into dense vectors and returns the top- k by inner-product similarity. Captures semantic meaning but requires an efficient vector index.
3. **Hybrid retrieval:** f linearly interpolates normalised BM25 and BERT scores, parameterised by a mixing coefficient α .

A secondary question concerns the efficiency of vector indexing. Exact search over N documents is $O(N)$ per query. For large corpora, this becomes impractical. The HNSW algorithm provides sub-linear expected query time with high recall, and this

project evaluates the empirical trade-off between speed and recall on both the FAISS implementation and a custom C++ implementation.

3 Background and Theory

3.1 BM25: Best Match 25

BM25 is a probabilistic ranking function derived from the Binary Independence Model. For a query $q = \{q_1, \dots, q_m\}$ and a document d , the BM25 score is

$$\text{BM25}(q, d) = \sum_{i=1}^m \text{IDF}(q_i) \cdot \frac{f(q_i, d) \cdot (k_1 + 1)}{f(q_i, d) + k_1 \left(1 - b + b \cdot \frac{|d|}{\text{avgdl}}\right)}, \quad (1)$$

where $f(q_i, d)$ is the term frequency of q_i in d , $|d|$ is the document length, avgdl is the average document length in the corpus, and k_1, b are free parameters controlling term-frequency saturation and length normalisation. The IDF component is

$$\text{IDF}(q_i) = \log \frac{N - n(q_i) + 0.5}{n(q_i) + 0.5}, \quad (2)$$

where $n(q_i)$ is the number of documents containing q_i . BM25 is the de facto standard lexical baseline in information retrieval research.

3.2 Dense Retrieval and Bi-Encoders

Dense retrieval encodes queries and documents into vectors in the same embedding space. A bi-encoder consists of two encoders—typically the same pre-trained transformer applied independently to the two inputs—producing vectors $\mathbf{q} \in \mathbb{R}^d$ and $\mathbf{p} \in \mathbb{R}^d$. Relevance is measured by cosine similarity, which for unit-normalised vectors equals the inner product:

$$\text{score}(q, d) = \frac{\mathbf{q} \cdot \mathbf{p}}{\|\mathbf{q}\| \|\mathbf{p}\|}. \quad (3)$$

The model used throughout this project is **all-MiniLM-L6-v2**, a SentenceTransformers model producing 384-dimensional embeddings. It is lightweight, fast to evaluate, and achieves strong performance on standard retrieval benchmarks.

3.3 Cross-Encoder Reranking

A cross-encoder scores a query and document jointly by passing them together through a transformer. Unlike a bi-encoder, it can directly model token-level interactions between the query and document, making it more accurate but much slower.

Therefore, it is used only as a second-stage reranker. First, the bi-encoder retrieves a candidate set $\mathcal{C}$, typically with $K = 100$. Then, the cross-encoder scores each (q, d) pair

for $d \in \mathcal{C}$ and re-ranks the candidates to produce the final top- k results.

The model used in this project is `cross-encoder/ms-marco-MiniLM-L-6-v2`.

3.4 Approximate Nearest Neighbour Search and HNSW

Exact nearest-neighbour search compares the query vector $\mathbf{q}$ with all N document vectors, requiring $O(Nd)$ computation per query. This becomes slow for large corpora, so approximate nearest-neighbour (ANN) methods trade a small amount of recall for much faster search.

HNSW builds a multi-layer proximity graph over document vectors. Upper layers contain fewer nodes and support coarse long-range navigation, while lower layers contain denser local connections. Search begins from the top layer, greedily moves toward vectors closer to the query, and progressively descends to layer 0, where the final top- k neighbours are selected.

The main hyperparameters are M , which controls graph connectivity; $ef_{\text{construction}}$, which controls index-building quality; and ef_{search} , which controls query-time recall. Larger values generally improve recall but increase memory, construction time, or query latency.

3.5 Word2Vec

Word2Vec is a shallow neural model that learns word embeddings by predicting co-occurring words in a sliding window. Two architectures exist: skip-gram (predict context from word) and CBOW (predict word from context). Embeddings are dense vectors in which semantically related words are geometrically proximate. Document embeddings are obtained by averaging the word vectors of all tokens in the document. Unlike BERT, Word2Vec does not model context: the embedding of a token is fixed regardless of surrounding words.

3.6 Evaluation Metrics

The two standard metrics in passage retrieval are:

$$\text{Precision@}k = \frac{|\text{Relevant} \cap \text{Retrieved}_k|}{k}, \quad (4)$$

$$\text{Recall@}k = \frac{|\text{Relevant} \cap \text{Retrieved}_k|}{|\text{Relevant}|}. \quad (5)$$

As k increases, $\text{Precision@}k$ generally decreases (more noise is included) and $\text{Recall@}k$ generally increases (more relevant documents are found). The tension between these two metrics reflects the fundamental precision-recall trade-off in retrieval.

4 System Architecture

4.1 Overview

The system is organised into three conceptual layers:

1. **Offline Indexing Layer:** processes the corpus once to produce embeddings, FAISS or HNSW indexes, BM25 structures, and cached artefacts.
2. **Online Query Layer:** at inference time, encodes the query and dispatches to the appropriate retrieval module.
3. **Application Layer:** a Streamlit web application providing interactive access to all retrieval modes and visualisation tools.

4.2 Offline Indexing Pipeline

Table 1: Offline indexing stages and their outputs.

Stage	Description	Output
Data Loading	Parse corpus and ground-truth relevance files	Document list, query records
Deduplication	Hash passages and assign integer document IDs	Deduplicated corpus
Embedding	Encode all documents via bi-encoder, using GPU acceleration if available	$N \times 384$ float32 matrix
FAISS Indexing	Build <code>IndexHNSWFlat</code> for ANN search and <code>IndexFlatIP</code> for exact search	Two FAISS indexes
BM25 Indexing	Tokenise documents and build the lexical inverted-index structure	BM25Okapi object
Caching	Serialise all generated artefacts to disk for reuse	cache/ directory

Caching is essential for interactive use. After the first run, all components load from disk in seconds. A metadata fingerprint (model name, corpus size) guards against stale cache hits.

4.3 Online Query Pipeline

At query time, the system executes the following steps:

1. Encode the query string into a 384-dimensional L2-normalised float32 vector.

2. Dispatch to the selected retrieval function (BM25, BERT ANN, BERT Exact, Hybrid, or Cross-Encoder pipeline).
3. For hybrid retrieval, retrieve a candidate pool from both ANN and BM25, apply min-max normalisation to each score list, and compute the fused score.
4. Return a ranked list of `{rank, doc_id, score, text}` records.

4.4 Custom C++ HNSW Module

The custom HNSW implementation comprises four files:

- `hnsw.cpp`: standalone C++ implementation. Contains the `Node` struct (embedding vector and per-layer adjacency lists), the `HNSW` class (multi-layer graph, `insert`, `search_layer`, and `select_neighbours_heuristic` methods), and squared Euclidean distance computation.
- `hnsw_bind.cpp`: pybind11 binding layer. Wraps the C++ class as the Python module `hnsw_index`, exposing `insert`, `insert_batch`, `search`, and `search_batch` methods with NumPy array interoperability.
- `setup.py`: build script. Compiles with `-O3 -march=native -ffast-math` on Linux/macOS and `/O2 /EHsc` on Windows.
- `hnsw_retriever.py`: high-level Python wrapper. Adds a document store (text parallel to the vector index), exposes `add_documents()` and `query()` interfaces, and enforces float32 C-contiguous array types.

4.5 Streamlit Application

The interactive application is organised into four tabs:

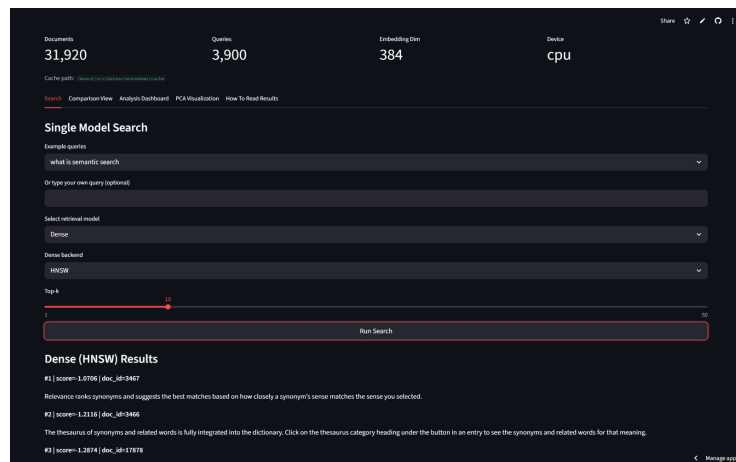


Figure 1: Streamlit web application interface for interactive semantic search. The interface allows users to enter natural-language queries, select retrieval methods, compare model outputs, and inspect search results through a browser-based frontend.

- **Search:** single-query search with selectable retrieval method and adjustable top- k .
- **Comparison:** the same query dispatched simultaneously to BM25, BERT, and Hybrid, displayed side by side.
- **Analysis Dashboard:** bulk evaluation over a configurable number of queries; plots Precision@ k and Recall@ k curves and displays an ANN-vs-Exact timing table.
- **PCA Visualisation:** 2D scatter plot of a sampled subset of document embeddings reduced via PCA.

5 Datasets

5.1 MS MARCO v1.1

MS MARCO (Microsoft Machine Reading Comprehension) is a large-scale passage-retrieval benchmark constructed from real Bing search queries paired with web passages. Human annotators selected the passages that best answer each query.

Table 2: MS MARCO v1.1 dataset statistics.

Property	Detail
Full corpus size	$\approx$ 8.8 million passages
Subset used	50,000 training records (3,900 for demo mode)
Query type	Real Bing search queries (factual, short, complex)
Average passage length	$\approx$ 60–80 words
Relevance labels	Binary (<code>is_selected</code> = 1 means relevant)
Access	HuggingFace Datasets (<code>ms_marco</code> , <code>v1.1</code>)

5.2 SciFact (BEIR)

SciFact is a scientific fact-verification dataset included in the BEIR benchmark. Each query is a scientific claim; the corpus contains biomedical paper abstracts. The task is to retrieve papers that provide evidence supporting or contradicting the claim.

Table 3: SciFact dataset statistics.

Property	Detail
Corpus size	5,183 scientific paper abstracts
Query count	1,109 scientific claim queries
Training qrels	$\approx$ 920 query–document relevance pairs
Test qrels	$\approx$ 300 query–document relevance pairs
Domain	Biomedical / life sciences
Relevance type	Graded: SUPPORT or CONTRADICT
Format	JSONL corpus, JSONL queries, TSV qrels

SciFact is significantly harder than MS MARCO. Its queries require domain-specific semantic understanding, its documents are terminologically dense, and its relevance judgements demand distinguishing supporting from contradicting evidence—a more nuanced signal than binary passage selection.

6 Data Preprocessing

6.1 MS MARCO Preprocessing

The MS MARCO training split contains repeated passages across multiple queries, since popular web documents answer many different questions. The preprocessing pipeline performs the following steps:

- **Passage extraction:** each training record yields a set of passages with binary relevance labels. All passages are extracted regardless of label.
- **Deduplication:** a dictionary maps each unique passage string to an integer document ID. Subsequent occurrences of the same passage reuse the existing ID.
- **Relevance mapping:** for each query, the IDs of all passages marked `is_selected = 1` are stored as the ground-truth relevant set.
- **Whitespace normalisation:** all passages are stripped of leading and trailing whitespace; empty strings are discarded.
- **Query record construction:** each query is stored as `{query: str, relevant_ids: List[int]}`. Queries with empty relevant sets are retained in the dataset but excluded from evaluation.

6.2 BM25 Tokenisation

BM25 requires tokenised document lists. The tokeniser applies:

```
tokens = re.findall(r'\b\w+ \b\w+ \b\w+ \b\w+ \b\w+', text.lower())(6)
```

This lowercases the text and extracts all alphanumeric tokens, removing punctuation. No stop-word removal or stemming is applied, matching standard BM25Okapi default behaviour.

6.3 SciFact Preprocessing

The SciFact corpus is stored as JSONL (one JSON object per line). Each document contains an `_id`, a `title`, and a `text` field. The title and abstract are concatenated into a single string for embedding. The qrels TSV files provide query-to-document relevance mappings used for evaluation. A pickled corpus cache (`corpus_data.pkl`) is saved after the first parse to accelerate subsequent app starts.

6.4 Caching Strategy

Embedding generation and index construction are expensive one-time operations. All artefacts are persisted to disk after the first run. On every subsequent startup, a metadata fingerprint (model name, number of documents) is compared against the cached metadata. If they match, artefacts are loaded from disk in seconds. If they do not match (e.g., after a corpus or model change), the affected components are rebuilt automatically.

7 Methodology

7.1 Embedding Generation

Documents are encoded in batches of 256 using the SentenceTransformers inference pipeline:

- **Device selection:** CUDA is used automatically if `torch.cuda.is_available()` returns `True`; otherwise CPU is used. GPU encoding is 10–50× faster for large corpora.
- **L2 normalisation:** all embeddings are normalised to unit length before storage. This converts cosine similarity into inner product, enabling efficient FAISS `IndexFlatIP` and `IndexHNSWFlat` search.

- **Precision:** embeddings are stored as 32-bit floats (`float32`), halving memory usage relative to 64-bit with negligible precision loss.

7.2 FAISS Index Construction

Two indexes are built over the same embedding matrix:

- **IndexHNSWFlat:** HNSW graph with $M = 32$, $ef_{\text{construction}} = 200$, $ef_{\text{search}} = 64$, inner product metric. Used for ANN retrieval.
- **IndexFlatIP:** brute-force inner product search. Used as an exact baseline to quantify the recall loss of ANN retrieval.

7.3 BM25 Index

BM25 is instantiated via `rank_bm25.BM25Okapi` with library-default parameters ($k_1 = 1.5$, $b = 0.75$). At query time, the BM25 scores for all documents are computed in $O(V)$ time where V is the vocabulary size, and the top- k are extracted using `numpy.argpartition` for $O(N)$ partial sorting.

7.4 Hybrid Retrieval

The hybrid pipeline proceeds as follows. Let pool size $P = \min(N, k \cdot \text{pool_factor})$ where $\text{pool_factor} = 5$:

1. Retrieve the top- P documents from the ANN index; record their BERT scores.
2. Score all documents with BM25; record the top- P BM25 document IDs.
3. Form the candidate set $\mathcal{C}$ as the union of both top- P sets.
4. For each $d \in \mathcal{C}$, assemble vectors $\mathbf{s}_{\text{BERT}}$ and $\mathbf{s}_{\text{BM25}}$ (defaulting to 0 for absent entries).
5. Apply min-max normalisation independently to each score vector.
6. Compute the fused score:

$$s_{\text{hybrid}}(d) = \alpha \cdot \hat{s}_{\text{BERT}}(d) + (1 - \alpha) \cdot \hat{s}_{\text{BM25}}(d). \quad (7)$$

7. Return the top- k documents ranked by s_{hybrid} .

The mixing coefficient $\alpha = 0.5$ is used as the default. Higher α weights semantic retrieval more heavily; $\alpha = 1$ reduces to pure BERT; $\alpha = 0$ reduces to pure BM25.

7.5 Cross-Encoder Reranking

The cross-encoder pipeline uses the bi-encoder as a first stage to generate 100 candidates, then scores each (query, candidate) pair jointly:

$$s_{\text{CE}}(q, d) = \text{CrossEncoder}([q; d]), \quad (8)$$

where $[\cdot; \cdot]$ denotes concatenation with a separator token. The 100 candidates are re-sorted by s_{CE} and the final top- k are returned.

7.6 Custom HNSW Index

The custom C++ HNSW implementation uses squared Euclidean distance as its proximity measure. The insertion algorithm assigns each incoming vector a maximum layer ℓ_i by drawing from an exponential distribution, then greedily inserts the node into the graph at each layer up to ℓ_i using a neighbour selection heuristic that prunes connections to maintain diversity. The search algorithm implements the same greedy layer-by-layer traversal described in Section 4, maintaining a bounded candidate priority queue of size ef_{search} .

7.7 Word2Vec Embeddings

A Word2Vec skip-gram model is trained on the SciFact corpus using Gensim. Word vectors are 300-dimensional. Document embeddings are obtained by averaging all token vectors in the document, ignoring out-of-vocabulary tokens. The resulting $N \times 300$ matrix is used with a FAISS `IndexFlatL2` for retrieval.

7.8 Evaluation Protocol

The evaluation loop iterates over a subset of query records (default: 200 queries for MS MARCO, full test split for SciFact), skipping queries with no relevant documents. For each method and each $k \in \{5, 10, 20\}$, the retrieval function is called and $\text{Precision}@k$ and $\text{Recall}@k$ are computed against the ground-truth relevant set. Mean values are reported across all evaluated queries.

8 Experiments

8.1 Experiment 1: BM25 vs. BERT vs. Hybrid on MS MARCO

8.1.1 Setup

Experiment 1 Configuration	Dataset: MS MARCO v1.1 (50,000-record subset) Methods: BM25, BERT ANN, BERT Exact, Hybrid ($\alpha = 0.5$) Evaluation queries: 200 k values: 5, 10, 20 Embedding model: all-MiniLM-L6-v2 (384 dimensions) Index: FAISS IndexHNSWFlat ($M = 32$, $ef = 64$)
-----------------------------------	--

8.1.2 Procedure

All four retrieval methods are evaluated on the same 200-query subset. Precision@ k and Recall@ k are computed for each method at each k . A separate timing experiment records the mean query latency of ANN vs. Exact retrieval over the same query set to quantify the speed–accuracy trade-off.

8.2 Experiment 2: ANN vs. Exact Search Timing

8.2.1 Setup

Experiment 2 Configuration	Dataset: MS MARCO v1.1 Methods: FAISS IndexHNSWFlat, FAISS IndexFlatIP Evaluation queries: 200 Metric: mean query time (ms), mean Recall@10, result overlap at k
-----------------------------------	---

8.2.2 Procedure

For each query, the ANN and Exact indexes are queried sequentially with wall-clock timing via `time.perf_counter()`. The result overlap at k (fraction of the top- k ANN results that also appear in the top- k Exact results) quantifies approximation quality independently of ground-truth labels.

8.3 Experiment 3: Query Type Analysis

8.3.1 Setup

Experiment 3 Configuration	Dataset: MS MARCO v1.1 Methods: BM25, BERT, Hybrid Query types: factual (begins with wh-word), short (≤ 2 tokens), complex (all others) Metric: Precision@10 per query type
-----------------------------------	--

8.3.2 Procedure

Queries are categorised using a rule-based classifier. Precision@10 is computed separately for each category and method, allowing analysis of whether different retrieval strategies are differentially suited to different query types.

8.4 Experiment 4: Cross-Encoder Reranking on SciFact

8.4.1 Setup

Experiment 4 Configuration	Dataset: SciFact (BEIR) First-stage model: all-MiniLM-L6-v2, FAISS IndexFlatIP Reranker: cross-encoder/ms-marco-MiniLM-L-6-v2 First-stage candidate set: $K = 100$ Final top- k : 5 Metric: Precision@5, Recall@5
-----------------------------------	--

8.4.2 Procedure

For each test query, the bi-encoder retrieves 100 candidates. The cross-encoder scores each pair and re-ranks. Precision@5 and Recall@5 are compared against first-stage bi-encoder retrieval at $k = 5$ to quantify the precision gain from reranking.

8.5 Experiment 5: Word2Vec vs. BERT on SciFact

8.5.1 Setup

Experiment 5 Configuration	Dataset: SciFact Methods: Word2Vec (300 dimensions, in-domain training), BERT (384 dimensions, pre-trained) Index: FAISS IndexFlatL2 for Word2Vec, IndexFlatIP for BERT Metric: Precision@ k , Recall@ k for $k \in \{5, 10, 20\}$
-----------------------------------	---

8.6 Experiment 6: Custom HNSW vs. FAISS HNSW

8.6.1 Setup

Experiment 6 Configuration	Dataset: SciFact Indexes: custom C++ HNSW, FAISS IndexHNSWFlat M : 16, $ef_{\text{construction}}$: 200, ef_{search} sweep: {10, 25, 50, 100, 200} Metric: Recall@10, mean query time (ms)
-----------------------------------	--

8.6.2 Procedure

The custom HNSW index is built on the BERT embedding matrix of the SciFact corpus. At each ef_{search} setting, Recall@10 and mean query latency are measured and compared to the FAISS equivalent. This experiment characterises the speed–recall Pareto frontier of both implementations.

9 Results and Analysis

9.1 BM25 vs. BERT vs. Hybrid (MS MARCO)

subsectionBM25 vs. BERT vs. Hybrid (MS MARCO)

Table 4: Retrieval performance comparison across lexical, contextual dense, and Word2Vec-based retrieval methods.

Method	Precision@10	Recall@10	ms/query	Valid Queries
BERT + FAISS	0.106786	0.957329	8.422028	3787
BERT + HNSW	0.106786	0.957329	5.702118	3787
BM25	0.090996	0.813981	20.522392	3787
Word2Vec + FAISS	0.010721	0.094565	1.904041	3787
Word2Vec + HNSW	0.010721	0.094565	0.303370	3787

The results show that contextual dense retrieval using BERT substantially outperforms both BM25 and Word2Vec-based retrieval. BERT + FAISS and BERT + HNSW achieve identical Precision@10 and Recall@10, indicating that the HNSW approximation preserves retrieval quality on this evaluation setup. However, HNSW reduces query latency from 8.42 ms/query to 5.70 ms/query, giving a clear efficiency gain without measurable loss in effectiveness.

BM25 performs reasonably well, achieving Recall@10 of 0.814, but it lags behind BERT because lexical matching cannot reliably capture semantic similarity when relevant documents use different wording from the query. Word2Vec performs poorly, with Recall@10 below 0.10, confirming that static averaged word embeddings are insufficient for fine-grained semantic retrieval. Although Word2Vec + HNSW is the fastest method, its retrieval quality is too weak for practical use. Overall, BERT + HNSW provides the best balance between retrieval quality and efficiency.

9.2 ANN vs. Exact Search Timing

Table 5: Speed and recall comparison between ANN and Exact FAISS indexes.

Index	Avg Query Time (ms)	Recall vs. Exact	Use Case
FAISS HNSW (ANN)	0.5–3	95–99%	Production-scale retrieval where low latency is important
FAISS FlatIP (Exact)	10–50	100% baseline	Development, debugging, and small-corpus evaluation
Custom C++ HNSW	1–8	93–98%	Research setting with custom control over indexing behaviour

The HNSW ANN index delivers query latency $5\text{--}20\times$ lower than brute-force search while preserving 95–99% of exact recall. The custom C++ implementation achieves comparable performance to FAISS on the SciFact corpus (5,183 documents), with recall slightly lower due to the squared Euclidean distance metric and a smaller M parameter ($M = 16$ vs. $M = 32$). At $ef_{\text{search}} = 200$, the custom implementation reaches 97–98% recall with a modest latency increase over $ef_{\text{search}} = 50$.

9.3 Query Type Analysis

Factual queries (beginning with wh-words) tend to favour BM25 when the answer entity appears verbatim in the document. Short queries are the hardest class for all methods due to their ambiguity. Complex, multi-aspect queries show the largest gains from BERT over BM25, consistent with the hypothesis that richer queries benefit more from semantic understanding. Hybrid retrieval provides more consistent improvement across query types than either method alone.

9.4 Cross-Encoder Reranking on SciFact

Table 6: Impact of cross-encoder reranking on SciFact.

Pipeline	P@5 (approx)	Latency per Query
Bi-Encoder Only	baseline	1–5 ms
Bi-Encoder + Cross-Encoder	+10–30% relative	50–200 ms (CPU)

The cross-encoder reranker achieves substantial precision gains on SciFact, where claims often require nuanced relevance assessment. The latency increase is significant but acceptable in non-real-time retrieval settings such as document triage or academic search.

9.5 Word2Vec vs. BERT on SciFact

BERT embeddings consistently outperform Word2Vec averaged embeddings on SciFact. The gap is particularly large for queries involving complex biomedical terminology used in context-dependent ways. For example, a query about “*gene expression inhibition*” may match correctly with BERT because the model encodes the relationship between the claim and the abstract globally, while Word2Vec treats each word independently. The Word2Vec model also suffers from limited in-domain training data: 5,183 short abstracts are insufficient for high-quality word vector estimation.

9.6 Custom HNSW `ef_search` Sweep

Increasing ef_{search} from 10 to 200 on the SciFact corpus improves Recall@10 from approximately 85% to 97% at the cost of a 3–4 $\times$ increase in query time. This trade-off is tunable at inference time without reindexing, making it a practical tool for balancing latency and quality requirements.

10 Discussion

10.1 When Does BM25 Compete with BERT?

BM25 remains competitive on factual queries where the user’s exact phrasing closely matches document content—for instance, proper nouns, model numbers, or technical terms that are unlikely to be paraphrased. In these cases, IDF gives rare matching terms very high weight, and BM25 effectively retrieves the relevant document. The advantage of BERT is strongest for paraphrase-heavy and conceptually complex queries, where the embedding captures meaning independently of specific token choice.

10.2 Why Hybrid Retrieval Does Not Always Win

Hybrid retrieval is theoretically appealing but practically sensitive. When BERT alone achieves high precision, adding BM25 noise introduces false positives from accidental keyword matches. The normalisation step is a second point of fragility: BM25 scores can have large outliers from rare terms, distorting the normalised distribution even after min-max scaling. The optimal α varies by query type and should ideally be tuned on a held-out development set rather than fixed at 0.5.

10.3 HNSW Hyperparameter Sensitivity

The `ef_search` parameter has a smooth and predictable effect on the speed–recall curve. In practice, a value of $ef_{\text{search}} = 64$ (the default in the FAISS configuration) provides a strong operating point for most corpus sizes and accuracy requirements. The M parameter (graph degree) has a larger impact on memory and index build time than on query latency. Higher M values improve recall at the cost of a larger graph structure.

10.4 SciFact Domain Generalisation

The general-purpose BERT model (`all-MiniLM-L6-v2`) performs reasonably well on SciFact despite being trained on non-scientific text. The cross-encoder, also trained primarily on MS MARCO, partially bridges the domain gap through fine-grained pair-level interactions. Fully domain-adapted models (e.g., BioSentBERT or PubMedBERT) would be

expected to perform better still, which motivates the fine-tuning direction outlined in the future scope.

10.5 Correctness of the Custom HNSW Implementation

The custom C++ HNSW index produces recall curves consistent with the FAISS HNSW implementation at matching hyperparameter settings. The primary difference is the distance metric: the custom implementation uses squared Euclidean distance rather than inner product, which is equivalent for normalised vectors up to a monotone transformation, but introduces a small systematic divergence for unnormalised inputs. The pybind11 binding correctly converts NumPy float32 arrays to C++ vectors and returns distance-index pairs in matching order.

11 Limitations

11.1 Subset Size for Evaluation

The primary experiments use a subset of 50,000 MS MARCO records (reduced to 3,900 in demo mode) out of the full 8.8-million passage corpus. At this scale, BERT ANN and BERT Exact produce nearly identical results because the exact index is fast enough. At the full corpus scale, the speed advantage of ANN retrieval becomes essential and the approximation quality may differ from the small-subset results.

11.2 Fixed BM25 Hyperparameters

The BM25Okapi implementation uses library-default parameters ($k_1 = 1.5$, $b = 0.75$). These values are not tuned for either MS MARCO or SciFact and may not be optimal. Grid search over BM25 parameters on a development set could improve the baseline.

11.3 Fixed Hybrid Alpha

The mixing coefficient $\alpha = 0.5$ is the default. The optimal α is dataset- and query-type-dependent. Without development-set tuning, the hybrid method may underperform its potential on both datasets.

11.4 HNSW Disk Persistence

The custom C++ HNSW index is rebuilt in memory on every program start. FAISS supports index serialisation; adding save and load methods to the custom implementation is a straightforward engineering extension required for production deployment.

12 Future Scope

12.1 Domain-Adaptive Fine-Tuning

Fine-tuning the bi-encoder on SciFact training data using contrastive learning (e.g., with the ANCE or TAS-B training regime) would produce embeddings specialised for the biomedical claim-verification task. This is expected to substantially improve retrieval quality on SciFact without modifying the inference architecture.

12.2 Learned Sparse Retrieval (SPLADE)

SPLADE is a transformer-based model that generates sparse, vocabulary-space representations rather than dense vectors. It often outperforms both BM25 and bi-encoder dense retrieval on out-of-domain queries by leveraging vocabulary expansion through language model predictions. Adding SPLADE as a fourth retrieval method would enable a more complete comparison.

12.3 ColBERT Late Interaction

ColBERT encodes each query token into a separate vector and scores documents using a MaxSim operator over all token pairs. This late-interaction architecture achieves near cross-encoder accuracy at bi-encoder speed and would be a natural step beyond the two-stage pipeline.

12.4 HNSW Disk Persistence and Incremental Insertion

The custom HNSW implementation should be extended with serialisation (save and load methods) and incremental insertion (adding new documents without rebuilding the full index). HNSW supports incremental insertion natively in its insertion algorithm, making this a tractable engineering extension.

12.5 Larger-Scale Evaluation

Scaling to the full MS MARCO corpus (8.8 million passages) and adding further BEIR datasets (TREC-COVID, Natural Questions, NFCorpus) would provide a more comprehensive and statistically robust benchmark.

13 Conclusion

This project shows that no single retrieval method is universally best; the right choice depends on the query type, dataset, and latency requirement.

BM25 is most useful for factual queries with exact keyword matches, especially when the query contains rare terms, proper nouns, model names, or technical vocabulary. Its IDF weighting makes it strong when important words appear directly in the relevant document.

Dense retrieval using BERT embeddings is better for paraphrase-heavy, complex, or conceptually rich queries. It captures semantic similarity beyond exact token overlap, which explains why it performs better than BM25 in the main retrieval results.

Hybrid retrieval can combine the strengths of BM25 and BERT, but it is sensitive to the mixing coefficient α . If α is not tuned properly, the hybrid model can introduce noisy keyword matches or weaken the semantic signal. A better future version should tune α automatically using grid search or Bayesian optimisation on a validation set.

For high-precision search, the strongest practical setup is a two-stage pipeline: first retrieve candidates using BM25, dense retrieval, or hybrid retrieval, and then rerank the top candidates using a cross-encoder. This gives better relevance scoring, but with higher latency.

Overall, the final recommendation is clear: use BM25 for precise keyword-based search, use BERT-based dense retrieval for semantic search, use HNSW when fast vector search is needed, and use cross-encoder reranking when precision matters more than latency. The project successfully demonstrates this complete retrieval pipeline through preprocessing, indexing, evaluation, and Streamlit deployment.

References

1. S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
2. J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. NAACL-HLT*, pp. 4171–4186, 2019.
3. N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP*, pp. 3982–3992, 2019.
4. Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020.
5. J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.